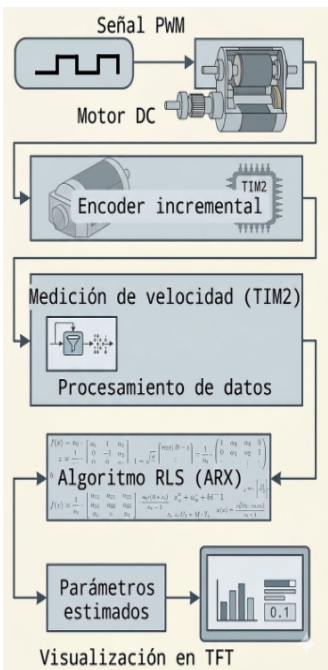


Módulo 4 - Generación PWM con TIM1 y TIM3

Proyecto de identificación RLS/ARX en STM32F405RGT6

Archivos documentados: pwm_driver.h, pwm_driver.c, pwm_if.h y pwm_if.c.



Función del módulo. Genera dos señales PWM distintas: una para accionar el motor DC y otra para controlar el brillo del display TFT.

Periféricos usados. TIM1_CH1 en PA8 para el PWM principal del motor y TIM3_CH3 en PB0 para el backlight del TFT.

Relación con el sistema. El motor recibe una entrada duty-cycle en permille desde la lógica principal, mientras que la pantalla usa un PWM separado para ajustar luminosidad sin interferir con el control del motor.

Patrón de diseño. El archivo pwm_if agrupa ambos canales bajo una sola interfaz constante, de modo que la aplicación invoca PWM_STM32.motor o PWM_STM32.tft_bl sin tocar registros directamente.

1. Papel del módulo dentro del firmware

En este proyecto la sigla PWM no se limita a una sola función. El módulo resuelve dos problemas diferentes con la misma técnica de modulación por ancho de pulso: regular la energía eléctrica entregada al motor y regular la luminosidad de la pantalla. Esa doble responsabilidad es útil para enseñar que un mismo recurso de temporización puede reutilizarse en subsistemas muy distintos del firmware.

Desde el punto de vista del control, el canal más importante es el del motor. La estimación RLS necesita excitar al sistema con una señal de entrada conocida; aquí esa entrada es el duty-cycle aplicado por TIM1. El segundo canal, el del backlight del TFT, no participa en la dinámica del motor, pero mejora la interacción con el usuario y muestra cómo separar un PWM funcional de un PWM de interfaz.

2. Relación entre archivos

Archivo	Responsabilidad
pwm_driver.h	Declara las funciones públicas del driver y la constante DUTY_MAX = 1000, con la que el proyecto expresa el ciclo de trabajo en permille.
pwm_driver.c	Configura los registros de GPIO y temporizadores, calcula prescalers y periodos, y actualiza registros CCR para modificar el duty.
pwm_if.h	Define una interfaz compuesta por dos submódulos: uno para el motor y otro para el backlight de la TFT.
pwm_if.c	Construye la instancia constante PWM_STM32 que conecta la aplicación con el driver real.

Fragmento 1. API publicada por pwm_driver.h

```
#define DUTY_MAX 1000u

void pwm_pb0_tim3_init(uint32_t pwm_hz, uint32_t duty_permille);
void pwm_pb0_tim3_set_duty(uint32_t duty_permille);
void pwm_pb0_tim3_invert_polarity(int invert);

void pwm_tim1_pa8_init(uint32_t pwm_hz);
void pwm_tim1_set_duty_permille(uint16_t duty_permille);
```

3. Mapa de hardware del módulo PWM

El diseño separa claramente los dos canales. TIM1 pertenece al bus APB2 y entrega la señal principal por PA8, mientras que TIM3 pertenece a APB1 y maneja el backlight por PB0. Esta decisión evita mezclar la temporización del motor con la del display y simplifica la depuración.

Canal	Timer	Pin	AF	Uso en el proyecto
Motor	TIM1_CH1	PA8	AF1	Señal PWM aplicada a la etapa de potencia del motor. En <code>main.c</code> se inicializa a 20 kHz.
Backlight TFT	TIM3_CH3	PB0	AF2	Controla el brillo del display. En el arranque se usa a 10 kHz con duty inicial de 450/1000.

3.1 La unidad elegida para el duty: permille

El proyecto no usa porcentajes enteros de 0 a 100, sino permille de 0 a 1000. Eso ofrece una resolución lógica diez veces mejor en la capa de aplicación y evita trabajar directamente con registros CCR en el resto del firmware. Por ejemplo, 250 significa 25.0%, 450 significa 45.0% y 1000 representa 100% nominal.

4. Canal PWM del backlight: TIM3 en PB0

La rutina `pwm_pb0_tim3_init()` construye un PWM clásico usando un temporizador general. Primero habilita los relojes de GPIOB y TIM3, luego cambia PB0 a función alterna AF2 y finalmente programa la base de tiempo para obtener la frecuencia deseada.

Fragmento 2. Núcleo de la configuración del backlight con TIM3

```
static uint32_t get_apb1_timer_clock_hz(void)
{
    uint32_t hclk = 168000000UL;
    ...
    uint32_t pclk1_hz = hclk_hz / pclk1_div;
    return (pclk1_div == 1) ? pclk1_hz : (2U * pclk1_hz);
}

void pwm_pb0_tim3_init(uint32_t pwm_hz, uint32_t duty_permille)
{
    if (pwm_hz == 0) pwm_hz = 1000;
    if (duty_permille > 1000) duty_permille = 1000;

    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;

    GPIOB->MODER &= ~(3U << (0U * 2U));
    GPIOB->MODER |= (2U << (0U * 2U));
    GPIOB->AFR[0] &= ~(0xFU << (0U * 4U));
    GPIOB->AFR[0] |= (0x2U << (0U * 4U));
    ...
}
```

4.1 Cálculo de reloj para TIM3

El helper `get_apb1_timer_clock_hz()` interpreta los prescalers de `RCC->CFGR`. En tu configuración global, `APB1 = 42 MHz`, pero los timers de ese bus corren al doble cuando el prescaler es distinto de 1; por ello `TIM3CLK = 84 MHz`. Esta distinción es fundamental para que el cálculo del periodo sea correcto.

Paso	Expresión	Resultado con 10 kHz
Clock del timer	$TIM3CLK = 2 \times PCLK1$	84 MHz
Tick objetivo	$target_tick = 1 \text{ MHz}$	1,000,000 Hz
Prescaler	$PSC = TIM3CLK / target_tick - 1$	83
Frecuencia de conteo	$tick = TIM3CLK / (PSC+1)$	1 MHz
Periodo	$ARR = tick / pwm_hz - 1$	99
Duty inicial	$CCR3 = (ARR+1) \times 450 / 1000$	45

4.2 PWM mode 1 y preload en CH3

Una vez calculados PSC, ARR y CCR3, el driver configura TIM3->CCMR2 para usar PWM mode 1 en el canal 3 y activa el preload OC3PE. Después habilita la salida con CC3E, habilita el preload global de ARR y dispara un UG para transferir los registros sombra al temporizador activo antes de arrancar el conteo.

Fragmento 3. Secuencia final de arranque del PWM en TIM3

```
TIM3->PSC = (uint16_t)psc;
TIM3->ARR = (uint16_t)arr;
TIM3->CCR3 = (uint16_t)(((arr + 1UL) * duty_permille) / 1000UL);

TIM3->CCMR2 &= ~(TIM_CCMR2_CC3S_Msk | TIM_CCMR2_OC3M_Msk);
TIM3->CCMR2 |= (6U << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE;
TIM3->CCER |= TIM_CCER_CC3E;
TIM3->CR1 |= TIM_CR1_ARPE;
TIM3->EGR |= TIM_EGR_UG;
TIM3->CR1 |= TIM_CR1_CEN;
```

4.3 Actualización de brillo e inversión de polaridad

La función `pwm_pb0_tim3_set_duty()` solo relee ARR y reescribe CCR3; por tanto cambiar el brillo cuesta muy poco tiempo de CPU. La función `pwm_pb0_tim3_invert_polarity()` altera el bit CC3P de CCER para adaptarse a hardware activo-alto o activo-bajo. Esa función no se usa todavía en `main.c`, pero deja el driver listo para otras variantes de pantalla o transistor de backlight.

Fragmento 4. Cambio de duty y selección de polaridad

```
void pwm_pb0_tim3_set_duty(uint32_t duty_permille)
{
    if (duty_permille > 1000) duty_permille = 1000;
    uint32_t arr = TIM3->ARR;
    TIM3->CCR3 = (uint16_t)(((arr + 1UL) * duty_permille) / 1000UL);
}

void pwm_pb0_tim3_invert_polarity(int invert)
{
    if (invert) TIM3->CCER |= TIM_CCER_CC3P;
    else      TIM3->CCER &= ~TIM_CCER_CC3P;
}
```

5. Canal PWM del motor: TIM1 en PA8

El segundo canal es el realmente crítico para el comportamiento dinámico del sistema. La función `pwm_tim1_pa8_init()` usa el timer avanzado TIM1 y entrega la señal en PA8. A diferencia de TIM3, aquí el duty no se establece en la inicialización: el canal arranca con 0% y luego la aplicación lo actualiza según la variable `g_motor.d`.

5.1 Preparación de PA8 como salida alternativa

La rutina pone PA8 en modo Alternate Function y asigna AF1, que corresponde a TIM1_CH1. También configura la salida como push-pull, sin resistencias internas y a muy alta velocidad. Esta última elección es razonable porque el flanco PWM debe ser limpio, sobre todo si la señal alimenta una etapa de potencia o un driver externo.

Fragmento 5. Configuración de PA8 para TIM1_CH1

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

GPIOA->MODER &= ~(3u << (8 * 2));
GPIOA->MODER |= (2u << (8 * 2));
GPIOA->AFR[1] &= ~(0xFu << ((8 - 8) * 4));
GPIOA->AFR[1] |= (0x1u << ((8 - 8) * 4));
GPIOA->OSPEEDR |= (3u << (8 * 2));
GPIOA->OTYPER &= ~(1u << 8);
GPIOA->PUPDR &= ~(3u << (8 * 2));
```

5.2 Cálculo del periodo en TIM1

El helper `get_tim1clk_hz()` parte del reloj de APB2. En tu configuración del sistema, $PCLK2 = 84 \text{ MHz}$ y, como el prescaler de APB2 es distinto de 1, el reloj efectivo de los timers de ese bus se duplica. Por tanto $TIM1CLK = 168 \text{ MHz}$.

A diferencia del canal de backlight, aquí el código intenta usar $PSC = 0$ siempre que el periodo quepa en 16 bits. Para una frecuencia de 20 kHz, sí cabe perfectamente. Eso maximiza la resolución temporal del duty porque el contador avanza a la mayor velocidad disponible.

Paso	Expresión	Resultado con 20 kHz
Clock del timer	$TIM1CLK = 2 \times PCLK2$	168 MHz
Prescaler elegido	$PSC = 0$	0
Periodo	$ARR = TIM1CLK / pwm_hz - 1$	8399
Resolución del duty	$ARR + 1$ niveles teóricos	8400 niveles
Duty inicial	$CCR1 = 0$	0% al arrancar

5.3 Particularidad de TIM1: el bit MOE

TIM1 es un temporizador avanzado. Por eso, además de habilitar CC1E, el código debe activar `BDTR.MOE`. Si este bit no se pone en 1, el contador puede correr internamente pero la salida física no aparece en el pin. Este detalle es clave para estudiantes: cuando un PWM de TIM1 o TIM8 no sale al exterior, uno de los primeros puntos a revisar es precisamente MOE.

Fragmento 6. Arranque del PWM del motor en TIM1

```
TIM1->PSC = (uint16_t)psc;
TIM1->ARR = (uint16_t)arr;
g_arr = (uint16_t)arr;
TIM1->CCR1 = 0;

TIM1->CCMR1 &= ~(TIM_CCMR1_OC1M);
TIM1->CCMR1 |= (6u << TIM_CCMR1_OC1M_Pos);
TIM1->CCMR1 |= TIM_CCMR1_OC1PE;
TIM1->CCER |= TIM_CCER_CC1E;
TIM1->CR1 |= TIM_CR1_ARPE;
TIM1->BDTR |= TIM_BDTR_MOE;
TIM1->EGR |= TIM_EGR_UG;
TIM1->CR1 |= TIM_CR1_CEN;
```

5.4 Actualización del duty del motor

La función `pwm_tim1_set_duty_permille()` convierte el valor lógico 0..1000 en un valor de CCR1 usando la variable global interna `g_arr`. El cálculo se hace en 32 bits para evitar desbordamientos durante la multiplicación y después se trunca a 16 bits al escribir en el registro.

Fragmento 7. Conversión de duty lógico a CCR1

```
void pwm_tim1_set_duty_permille(uint16_t duty_permille)
{
    if (duty_permille > DUTY_MAX) duty_permille = DUTY_MAX;

    uint32_t ccr = ((uint32_t)duty_permille * (uint32_t)(g_arr + 1u)) / DUTY_MAX;
    if (ccr > g_arr) ccr = g_arr;
    TIM1->CCR1 = (uint16_t)ccr;
}
```

5.5 Una observación fina sobre el 100% del motor

El canal del motor satura CCR1 a `g_arr`, no a `g_arr + 1`. Eso significa que el 1000 permille se traduce en un duty prácticamente máximo, pero matemáticamente queda apenas por debajo del 100% ideal. Para control de motor esto suele ser irrelevante, pero como material didáctico es interesante porque muestra cómo pequeñas decisiones de implementación afectan la interpretación exacta de la variable de usuario.

6. Capa de interfaz: pwm_if

La capa `pwm_if` empaqueta ambos canales en una sola estructura jerárquica. Esto ayuda a mantener la aplicación limpia: en vez de recordar nombres largos de funciones, el código principal accede a una sola constante y desde allí selecciona el subsistema que necesita. Es un ejemplo simple pero muy útil de abstracción en bare-metal.

Fragmento 8. Estructuras públicas de `pwm_if.h`

```
typedef struct {
    pwm_motor_init_fn_t init;
    pwm_motor_set_fn_t set_duty_permille;
} pwm_motor_if_t;

typedef struct {
    pwm_bl_init_fn_t init;
    pwm_bl_set_fn_t set_duty;
    pwm_bl_inv_fn_t invert_polarity;
} pwm_backlight_if_t;

typedef struct {
    pwm_motor_if_t motor;
    pwm_backlight_if_t tft_bl;
} pwm_if_t;
```

Fragmento 9. Instancia concreta construida en `pwm_if.c`

```
const pwm_if_t PWM_STM32 = {
    .motor = {
        .init = pwm_tim1_pa8_init,
        .set_duty_permille = pwm_tim1_set_duty_permille,
    },
    .tft_bl = {
        .init = pwm_pb0_tim3_init,
        .set_duty = pwm_pb0_tim3_set_duty,
        .invert_polarity = pwm_pb0_tim3_invert_polarity,
    }
};
```

7. Uso real del módulo en la aplicación

El archivo `main.c` deja claro el propósito de cada canal. Durante el arranque, el backlight se inicializa al preparar la pantalla; luego, tras iniciar el encoder, se prepara el PWM del motor a 20 kHz. Ya dentro del lazo principal, la señal del motor se actualiza constantemente con el duty almacenado en `g_motor.d`.

Fragmento 10. Uso del backlight y del PWM del motor desde main.c

```
void ini_pantalla(void)
{
    GPIO_STM32.tft.init();
    PWM_STM32.tft_bl.init(10000, 450);
    SPI1_STM32.gpio_init();
    SPI1_STM32.init();
    ...
}

int main(void)
{
    ...
    quad_STM32.init();
    PWM_STM32.motor.init(PWM_TARGET_HZ);
    ...
    while (1)
    {
        GPIO_STM32.motor.left();
        PWM_STM32.motor.set_duty_permille(g_motor.d);
        ...
    }
}
```

Además, la librería gráfica expone una función de conveniencia que vuelve a utilizar el canal del backlight sin que la capa de interfaz de usuario tenga que conocer detalles de timers o registros.

Fragmento 11. Ajuste de brillo encapsulado en ILI9341_driver.c

```
void ili_brightness(uint32_t brightness)
{
    PWM_STM32.tft_bl.set_duty(brightness);
}
```

8. Ideas didácticas y observaciones importantes

Punto	Explicación para clase
Separación de canales	El mismo concepto PWM sirve para dos fines distintos. Conviene explicar a los estudiantes que la técnica es la misma, pero el significado físico de la señal cambia según la carga.
Timers en APB1 y APB2	No basta con saber el clock del sistema; hay que distinguir entre PCLK y TIMCLK. Esa diferencia suele ser fuente de errores cuando la frecuencia medida no coincide con la calculada.
TIM1 es advanced	El bit MOE hace que TIM1 sea un buen ejemplo de cómo no todos los timers se inicializan exactamente igual.
Supuesto de 168 MHz	Los helpers de este módulo parten de $hclk = 168000000$. Si en otra práctica se cambia la configuración del reloj del sistema, las fórmulas del PWM quedarían incorrectas si no se actualiza ese supuesto.
Backlight con polaridad opcional	La función de inversión no se usa todavía, pero es buena para discutir compatibilidad con hardware activo-bajo y adaptación de firmware sin reescribir el resto de la aplicación.

9. Conclusión

El módulo PWM está bien planteado para un curso de firmware embebido porque combina configuración de pines, interpretación de árboles de reloj, programación de timers y diseño de interfaces. Además, conecta de forma muy clara con el objetivo global del proyecto: entregar una excitación controlada al motor y, al mismo tiempo, mantener una visualización cómoda en la pantalla TFT. También es un excelente bloque didáctico para explicar duty-cycle, configuración de pines en AF, diferencias entre timers generales y avanzados, y la relación entre frecuencia del bus, prescaler, preload y periodo.